# T4.1 Proyecto: gestor de entrenamientos

Un programa de consola para crear planes de entrenamiento y gestionarlos desde un menú. Es un único proyecto, individual, repartido en cinco archivos. No se pide en pasos ordenados: el enunciado describe las funciones del programa, que se pueden hacer en el orden que se quiera, y el programa completo, que las usa desde un menú.

Se ejecuta con Node.js y el paquete prompt-sync, como en clase:

```
// Importa el paquete prompt-sync
const prompt = require('prompt-sync')();

// Pide al usuario que ingrese su nombre
var nombre = prompt('¿Cuál es tu nombre? ');

// Saluda al usuario
console.log(`Hola, ${nombre}! Bienvenido al programa.`);
```

## Archivos

El código se organiza en estos archivos, que se cargan entre sí con `require('./datos')`:

- `datos.js`: estructuras de datos y funciones que las manipulan.
- `interfazUsuario.js`: interacción con el usuario, incluidos los menús.
- `app.js`: punto de entrada, coordina las demás partes.
- `funcionesUtiles.js`: funciones auxiliares.
- `estadisticas.js`: funciones que calculan estadísticas del plan.

Cada función del enunciado se exporta con `module.exports` desde el archivo donde está, con el nombre exacto y los parámetros en el orden del enunciado. Los tests buscan cada función entre lo que exportan todos los archivos salvo `app.js`, y prueban el programa ejecutándolo desde `app.js`. Se pueden añadir funciones auxiliares.

## Datos

Un plan es un objeto con su nombre y una lista de ejercicios; cada ejercicio, un objeto con nombre, duración en minutos e intensidad de 1 a 10:

```
{
  nombre: 'Plan de Fuerza',
  ejercicios: [
    { nombre: 'Sentadillas', duracion: 15, intensidad: 7 },
    { nombre: 'Dominadas', duracion: 10, intensidad: 5 },
  ],
}
```

## Inmutabilidad

Cuando el enunciado dice que una función no muta el plan, devuelve un plan nuevo con el cambio y no toca el que recibe (ni su lista, ni sus ejercicios). El cambio se guarda fuera de la función, sustituyendo el plan antiguo por el nuevo.

## Funciones

Cada función es una funcionalidad del programa, independiente de las demás, y se exporta con el nombre y los parámetros indicados.

### crearEjercicio

```
crearEjercicio(nombre: string, duracion: number, intensidad: number): Object
```

Crea un objeto ejercicio con los datos proporcionados y lo devuelve con el formato { nombre: 'Sentadillas', duracion: 15, intensidad: 7 }, sin más propiedades.

Los datos deben ser válidos: el nombre no puede estar vacío ni tener sólo espacios, la duración (en minutos) es un número positivo y la intensidad es un número entero entre 1 y 10, ambos incluidos. Si alguno no lo es (también si es NaN, como el que da Number("abc")), la función devuelve null y el menú muestra "Datos no válidos.".

### crearPlan

```
crearPlan(nombre: string): Object
```

Crea un nuevo plan de entrenamiento con el nombre indicado y lo devuelve con el formato { nombre: 'Plan de Fuerza', ejercicios: [] }: la lista de ejercicios empieza vacía y cada plan tiene la suya.

### agregarEjercicio

```
agregarEjercicio(plan: Object, ejercicio: Object): Object
```

Añade un ejercicio al final de la lista del plan y devuelve un nuevo objeto plan con él.

No muta el plan original: el plan devuelto es un objeto nuevo y su lista ejercicios es una instancia nueva. El cambio se guarda fuera de la función, sustituyendo el plan antiguo por el que se devuelve.

### eliminarEjercicio

```
eliminarEjercicio(plan: Object, nombreEjercicio: string): Object
```

Elimina del plan el ejercicio con ese nombre exacto (mayúsculas incluidas) y devuelve un nuevo objeto plan sin él; los demás conservan su orden. Si el ejercicio no existe, devuelve el plan sin cambios.

No muta el plan original.

### listarEjercicios

```
listarEjercicios(plan: Object): void
```

Muestra en consola el nombre y la duración de cada ejercicio del plan, uno por línea (un console.log por ejercicio), con el formato "Cardio: 30 minutos". No devuelve nada.

Si el plan no tiene ejercicios, muestra una única línea: "El plan no tiene ejercicios.".

### filtrarEjercicios

```
filtrarEjercicios(plan: Object, intensidad: number): Array<Object>
```

Devuelve un array con los ejercicios del plan cuya intensidad es igual o menor que la indicada, en el orden en que están. Si ninguno cumple, un array vacío.

No muta el plan original.

### calcularDuracionTotal

```
calcularDuracionTotal(plan: Object): number
```

Devuelve la suma de las duraciones de todos los ejercicios del plan, en minutos. Si el plan no tiene ejercicios, devuelve 0.

### actualizarEjercicio

```
actualizarEjercicio(plan: Object, nombreEjercicio: string, duracion: number,
intensidad?: number): Object
```

Actualiza la duración y, si se indica, la intensidad del ejercicio con ese nombre, y devuelve un nuevo objeto plan con el cambio. La intensidad es opcional: si no se pasa (undefined), se conserva la que tenía. Si el ejercicio no existe, devuelve el plan sin cambios.

No muta el plan original ni el objeto del ejercicio que se actualiza: el plan devuelto lleva un ejercicio nuevo en su lugar.

### buscarEjercicioRecursivo

```
buscarEjercicioRecursivo(plan: Object, nombreEjercicio: string): Object | null
```

Busca en el plan el ejercicio con ese nombre exacto (mayúsculas incluidas) y devuelve el objeto ejercicio, o null si no está.

Se implementa con recursividad: la función se llama a sí misma para recorrer la lista. No se pueden usar bucles (for, while) ni métodos que recorren la lista por ella (find, filter, some, indexOf…). Puede recibir un tercer parámetro opcional, por ejemplo la posición por la que va. No muta el plan.

### multiplicarDuracionesEjercicios

```
multiplicarDuracionesEjercicios(...duraciones: number[]): number
```

Devuelve el producto de todas las duraciones que recibe, sean cuantas sean (parámetro rest). Si no recibe ninguna, devuelve 1. Las duraciones 0 y negativas se multiplican sin más: con un 0 el producto es 0.

En el menú, la opción 8 multiplica las duraciones de todos los ejercicios del plan.

### ordenarEjercicios

```
ordenarEjercicios(plan: Object, criterio: 'nombre' | 'duracion' | 'intensidad'): Object
```

Devuelve un nuevo objeto plan con los ejercicios ordenados de forma ascendente según el criterio: por nombre, comparando con localeCompare; por duración o por intensidad, como números. Los ejercicios empatados conservan el orden que tenían. Con cualquier otro criterio, devuelve el plan sin cambios.

No muta el plan original (sort ordena el array sobre sí mismo).

### buscarEjerciciosPorNombre

```
buscarEjerciciosPorNombre(plan: Object, nombreParcial: string): Array<Object>
```

Devuelve un array con todos los ejercicios cuyo nombre contiene el texto indicado, sin distinguir mayúsculas y minúsculas, en el orden en que están. Si no hay ninguno, un array vacío.

Se usan funciones de orden superior como filter, sin bucles for ni while. No muta el plan.

### estadisticasPlan

```
estadisticasPlan(plan: Object): Object
```

Devuelve un objeto con tres propiedades, y nada más: duracionPromedio (la media de las duraciones, en minutos), intensidadPromedio (la media de las intensidades) y totalEjercicios (el número de ejercicios). Por ejemplo, { duracionPromedio: 35, intensidadPromedio: 6, totalEjercicios: 5 }. Las medias no se redondean.

Si el plan no tiene ejercicios, duracionPromedio e intensidadPromedio valen 0. No muta el plan.

### combinarPlanes

```
combinarPlanes(plan1: Object, plan2: Object): Object
```

Devuelve un nuevo plan llamado "Fuerza + Cardio" (los dos nombres unidos por " + ") con los ejercicios de plan1 y, a continuación, los de plan2.

Los nombres repetidos se renombran con un sufijo numérico: la primera aparición conserva el nombre y las siguientes pasan a ser "Remo2", "Remo3"…, también si el nombre se repite dentro de un mismo plan. Al renombrar sólo cambia el nombre.

No muta plan1 ni plan2, ni sus ejercicios: el renombrado se hace sobre ejercicios nuevos.

## El programa

app.js es el punto de entrada: carga las demás partes con require y arranca el menú principal. Los textos que se piden con prompt son libres; lo que se imprime con console.log se respeta al pie de la letra. Las líneas en blanco y los espacios a los lados de cada línea no cuentan.

Menú principal, que se repite hasta salir:

```
--- Menú Principal ---
1. Crear un nuevo plan
2. Gestionar un plan existente
3. Salir
```

1: pide el nombre del plan, lo crea con crearPlan, lo añade a la lista de planes y muestra "Plan creado.".

2: sin planes, muestra "No hay planes disponibles.". Si hay, muestra "Planes disponibles:" y una línea por plan, "1. Fuerza", "2. Cardio"…, y pide el número. Un número que no está en la lista: "Plan no válido." y vuelta al menú principal.

3: muestra "¡Hasta pronto!" y el programa termina.

Cualquier otra opción: "Opción no válida.".

Menú de un plan, que se repite hasta la opción 13:

```
--- Gestionando: Fuerza ---
1. Agregar ejercicio
2. Eliminar ejercicio
3. Listar ejercicios
4. Filtrar ejercicios por intensidad
5. Actualizar ejercicio
6. Calcular duración total del plan
7. Buscar ejercicio recursivamente
8. Multiplicar duraciones de ejercicios
9. Ordenar ejercicios
10. Buscar ejercicios por nombre parcial
11. Ver estadísticas del plan
12. Combinar con otro plan
13. Volver al menú principal
```

Las opciones que cambian el plan sustituyen el plan de la lista por el que devuelve la función. Cuando se muestra un ejercicio completo, el formato es "Remo: 20 minutos, intensidad 6".

1: pide nombre, duración e intensidad (convertidas con Number). Si crearEjercicio devuelve null, "Datos no válidos."; si no, lo añade y muestra "Ejercicio agregado.".

2: pide el nombre. Si no está, "No existe ningún ejercicio con ese nombre."; si está, lo elimina y muestra "Ejercicio eliminado.".

3: listarEjercicios.

4: pide la intensidad máxima y muestra cada ejercicio filtrado completo, o "Ningún ejercicio cumple el filtro.".

5: pide el nombre. Si no está, "No existe ningún ejercicio con ese nombre." sin pedir nada más. Si está, pide la nueva duración y la nueva intensidad; una intensidad vacía la deja como estaba. Con una duración no positiva o una intensidad que no sea un entero de 1 a 10, "Datos no válidos."; si no, "Ejercicio actualizado.".

6: "Duración total: 70 minutos".

7: pide el nombre y muestra el ejercicio completo, o "Ejercicio no encontrado.".

8: "Producto de las duraciones: 1200", con las duraciones de todos los ejercicios del plan.

9: pide el criterio. Si no es nombre, duracion ni intensidad, "Criterio no válido."; si lo es, ordena y muestra "Ejercicios ordenados.".

10: pide el texto y muestra cada ejercicio encontrado completo, o "No se han encontrado ejercicios.".

11: tres líneas: "Duración promedio: 35 minutos", "Intensidad promedio: 6" y "Total de ejercicios: 2".

12: sin otros planes, "No hay otros planes para combinar.". Si los hay, muestra "Planes disponibles:" con los demás planes (sin el actual) numerados desde 1 y pide el número; uno que no está, "Plan no válido.". El plan combinado se añade al final de la lista de planes y se muestra "Plan combinado creado: Fuerza + Cardio". El plan que se gestiona sigue siendo el mismo.

13: vuelve al menú principal.

Cualquier otra opción: "Opción no válida.".